

QUESTION 1:

The naive one is 1.97 gb while the multi-stage one is 556mb. The calculation for compressed percentage as follows gives: $(1.97\text{gb} - 0.556\text{gb}) / 1.97\text{gb} * 100 = 71.78\%$

When observed on what actually differs in between both:

Component	spam-api:naive	spam-api:multistage	Delta / What Changed
Total Size	1.97 GB	556 MB	~71.78% total reduction
C Compiler (gcc)	Present (/usr/bin/gcc)	Not found	Compilation toolchain left in builder stage
/usr/lib (System Libraries)	523 MB	47 MB	476 MB of development headers and shared objects shed
/usr/share (Docs, locales, assets)	238 MB	6.4 MB	231.6 MB of non-runtime assets excluded
Python Packages	297 MB (pip install)	288 MB (COPY /install)	Identical ML runtime capability preserved

QUESTION 2:

The speedup observed:

Request 1: Expecting Cache MISS

Status Code: 200

Response: {'label': 'spam', 'source': 'model'}

Latency: 32.94 ms

Speedup Factor: 8.94x faster on Cache Hit

Request 2: Expecting Cache HIT

Status Code: 200

Response: {'label': 'spam', 'source': 'cache'}

Latency: 3.69 ms

Docker is a single isolated package in a container, developers can't do multi container networking. Docker compose solves this by linking containers. But, docker compose is restricted to single host, but kubernetes operates at cluster scale.

QUESTION 3:

Parallelism: 4 . Given total completions: 8, total compute power= 2×2 CPUs= 4 CPUs
Say if we fixed parallelism as 8 then pod 0-3 would run cause the remaining wouldn't since only 4 CPUs are available. Say parallelism as 4 then 4 CPUs run pod 0-3 and once they are freed up, pods 4-7 would run. If mathematically, we can derive as: number of CPUs/number of CPUs needed for 1 pod, which also gives 4.

Why Kubernetes API Logs and not a Shared Volume : In kubernetes , for multiple pods to write to the same disk we indeed ReadWriteMany, but the default storage in minikube is ReadWriteOnce.

IF 6 CPUs are available, parallelism choice according to the above formula would be 6. And in that case, pods 0-5 in one round, and pods 6-7 in another while with parallelism 4, it would be 2 complete rounds. Thus with parallelism 6, since most of the batch would be done in the first round, changing parallelism would be good.

QUESTION 4:

For self healing: Replicaset controller is responsible for this, the way it works is it checks the actual observed state against the desired state which is 2 in this case and when manual deletion was done, it recognizes $1 < 2$ and immediately sets up new pod

If application deployed as a job instead of deployment: If it was job zero downtime wouldn't be possible since job specifications are immutable. So to upgrade it, running instances would have to be terminated entirely causing service downtime

Deployment is for APIs which have to run indefinitely, recover automatically and support zero downtime. Job is for when processes are expected for completion like say model training. And these are right fits in each case because shards was a validation task which needed completion.